

ROM Tracker Application Implementation Documentation

1. Introduction

This document explains the work completed on the **ROM Tracker** application from the Flutter/UI Integration team side. It describes the implemented phases, the issues found in the previous version, how they were fixed, what was implemented locally as Demo/Mock behavior, and what is still postponed to the actual Back-end and AI integration stages.

The purpose of this document is to: - Document the current project progress. - Justify the technical and design decisions made during implementation. - Clarify the difference between what is completed locally inside the app and what still requires real integration. - Support the final project discussion and presentation.

2. Project Idea Summary

ROM Tracker is a digital physical therapy follow-up application that connects:

- The patient - The doctor - The therapy tracking flow - The camera/AI module
- The Back-end

The app is designed to: - Allow the patient to book physical therapy sessions. - Track sessions, symptoms, and recovery progress. - Allow the doctor to manage sessions and patients. - Provide a future path for camera + AI motion analysis integration.

3. Adopted Implementation Approach

The following rules were adopted during implementation: - Full-screen UI images were used only as visual references. - Real implementation relied on the original assets located inside: - `assets/phase1` - `assets/phase2` - `assets/icons`

- No full screenshot was used as a ready-made UI screen. - Screens were built with real Flutter code as much as possible. - The UI/UX design was respected without inventing new flows or layouts.

4. Issues Found in the Previous Version

Before the modifications started, the previous app version contained several major issues: - Problems in the overall application architecture. - Some screens depended on full UI screenshots instead of real Flutter layouts. - Patient and doctor flows were not properly separated. - Authentication and authorization problems existed. - Incorrect logic was used to detect user type from the email address. - Many screens had overflow issues and yellow/black warning stripes. - Several images were placed incorrectly or forced into wrong placeholders. - Many elements were non-interactive or routed to the wrong destinations.

5. Phase One: Entry and Authentication

5.1 Implemented Screens

The entry and authentication flow was rebuilt in a logical way: - Splash - Onboarding - Auth Entry - Login - Forgot Password - OTP - User Types - Patient Sign Up Flow - Doctor Sign Up Flow

5.2 Adopted User Flows

Patient

- User Type
- Condition / Symptoms
- Age
- Basic Data
- Create Account

Doctor

- User Type
- Specialization
- Age
- Basic Data
- More Data
- Under Review

5.3 Fixes Applied

- Required fields can no longer be left empty.
- Email fields now require a valid email format.
- Phone fields now accept numbers only.
- Name fields accept letters only where appropriate.
- Login and Sign Up navigation was corrected.
- The startup screens and their assets were rebuilt and fixed.

5.4 Why Mock Authentication Was Used

`MockAuthService` was used because: - The Flutter team is responsible for the app flow and UI before real integration. - The real Back-end was not available yet. - At this stage, the priority was proving the correctness of the app flow rather than implementing real authentication.

6. Phase Two: Patient Flow

6.1 Implemented Screens

- Patient Home
- Search

- Symptoms
- Condition Details
- Camera UI Demo
- Top Doctors
- Doctor Details
- Patient Details
- Payment Methods
- Add Card
- Payment Success
- Sessions
- Chat
- Notifications
- Profile
- Edit Profile
- Settings
- Contact Us
- Wishlist
- Reviews

6.2 Implemented Patient Features

- The home page was made responsive and visually closer to the UI/UX reference.
- Search was activated inside Home, Symptoms, and Sessions.
- Top Doctors cards were connected to Doctor Details.
- Each doctor now has unique name, image, specialty, description, and close pricing.
- Booking data flows from Doctor Details to Patient Details to Payment.
- A logical Demo Payment flow was implemented using test credentials.
- After payment, the booked session is added locally to Upcoming Sessions.
- The following features were activated:
 - Cancel
 - Complete
 - Re-Book
 - Add/Edit Review
- A working Wishlist feature was added.
- A working Reviews feature was added.
- Local in-app Notifications were added.
- A red unread badge was added to the notification bell.

6.3 Why Demo Payment Was Used

Demo Payment was used instead of a real payment gateway because: - The project is still in the app-building and UI-flow stage. - Real payment requires Back-end + Payment Gateway integration. - During the discussion/demo, the flow can be shown correctly without requiring real financial processing. - This

allows booking, payment, and sessions to be tested before the final integration phase.

7. Phase Three: Doctor Flow

7.1 Implemented Screens

- Doctor Home
- Doctor Sessions
- Doctor Patient Details
- Doctor Chat
- Doctor Profile
- Doctor Wallet
- Doctor Notifications

7.2 Implemented Doctor Features

- Doctor sessions were separated from patient sessions.
- Doctor Sessions now includes:
 - Upcoming
 - Completed
 - Canceled
- The following actions were implemented:
 - Cancel Session
 - Complete Session
 - Restore / Reschedule
- A local logical Doctor Wallet was added.
- Doctor notifications were connected.
- Chat was linked locally between the doctor and patient accounts.

8. Local Sync Between the Two Accounts

8.1 Locally Linked Accounts

The following two accounts were linked locally: - patient@app.com - doctor@app.com

8.2 What This Local Sync Does

When the patient books a session with the linked doctor: - The session appears in the patient's Sessions. - A patient notification is created. - A doctor notification is created. - The session appears in Doctor Sessions. - A local wallet transaction appears in Doctor Wallet. - A local chat interaction appears between both accounts.

8.3 Why Local Demo Sync Was Used

Local Demo Sync was used because: - The current goal is to prove the app flow inside Flutter. - Real synchronization between patient and doctor requires a Back-end. - Local sync allows us to prove the complete scenario before introducing APIs.

9. Why This Local Sync Will Not Be Thrown Away Later

The current local sync does not mean we will rebuild the app from scratch later. When real integration starts, we will mainly replace: - The data source - The persistence layer - The communication logic between screens and services

However, we will most likely keep: - The UI screens - The navigation flow - Most of the app structure

So the current work is: - A practical prototype - An architectural base - A ready UI layer for real Back-end and AI integration later

10. What Requires a Real Back-end

The following features require a real Back-end later: - Real sign up and login - Storing users in a database - Fetching real doctors and patients - Real booking between patient and doctor - Synchronized session state updates - Real-time chat between accounts - Real notifications across devices - Real financial/payment flows - Persistent reviews and ratings

11. What Requires Real AI Integration

The AI-related part will later require: - Real device camera access - Capturing images, videos, or frames - Sending data to the AI service or receiving analysis results - Displaying the AI analysis output inside the application

12. Recommended Integration Order

The recommended sequence is: 1. Fully review the app locally. 2. Close both Patient Flow and Doctor Flow. 3. Align contracts with the Back-end team. 4. Integrate the real Back-end. 5. After Back-end stability, integrate camera + AI.

Why Back-end Should Come Before AI

Because the Back-end is responsible for: - users - sessions - chat - notifications - payments

The AI flow depends on having a stable therapy/session flow first.

13. Delayed or Frozen Features at the Current Stage

The following features are still demo-only or not fully completed:

1. Voice Search
 - Present visually only.
 - Reason: it needs a plugin, permissions, and real activation later.
2. Chat Attachments
 - Camera / Gallery / Location / Document / Mic are not truly integrated yet.
 - Reason: they need real implementation rather than UI placeholders.
3. Contact Us Actions
 - The icons are visible but do not yet open external apps.
 - Reason: no final external interaction flow was approved yet.
4. Real Wallet / Payment
 - Currently available as local demo logic only.
 - Reason: real transactions need Back-end and payment integration.
5. Real Push Notifications
 - The current implementation is in-app notifications only.
 - Reason: real push notifications require Back-end + a service such as Firebase.
6. AI Camera Integration
 - The camera screen currently exists as UI only.
 - Reason: the AI integration is postponed to the final stage.
7. Real Authentication
 - Authentication is still mock-based.
 - Reason: waiting for the actual Back-end integration.

Total postponed/frozen major features at the moment: **7 main features**

14. Evaluation of the Current App State

Strengths

- The application flow is now much more logical than the initial version.
- Patient and doctor flows are more clearly separated.
- The app is now suitable for demonstration and testing.
- The UI is much closer to the provided UI/UX than the original version.
- Many visual issues and overflow problems were removed.
- A nearly complete local demo exists for booking, payment, sessions, notifications, and chat.

Current Weaknesses

- Some parts of the app still rely on Mock/Demo behavior.
- There is still no real Back-end integration.
- There is still no real AI integration.
- Some secondary details may still need final polishing during the last review round.

15. Recommended Next Step

The recommended next step is: - Review the Doctor Flow completely, just as the Patient Flow was reviewed. - Perform one final full app review. - Then begin Back-end integration directly. - After the Back-end becomes stable, start the camera + AI integration.

16. Conclusion

A strong Flutter/UI application base has been built, and we now have: - A nearly complete Patient Flow locally - A locally connected Doctor Flow - A clear local sync between the two linked accounts - A solid foundation for later Back-end and AI integration

This means that the next stage is not a rebuild of the project, but rather the evolution of the current local demo version into a fully connected production-ready academic prototype.